Анатомия LoRA в LLM

https://gemini.google.com/app/b8b1e7795c394731

*User prompt: Мне тут пытались объяснить про LoRA , но я так и не понял , куда мы вставляем эти матрицы . Давай начнём с начала . Что представляет из себя LLM , где там атеншен слои и куда мы вставляем и что : # LoRA: Детальная анатомия — куда и как добавляются матрицы ## 1. Исходная точка: что мы имеем ДО добавления LoRA Возьмём конкретный слой из **Llama-7B** (архитектура очень показательна). Внутри каждого Transformer-блока есть линейные проекции: ``` Transformer Block (×32 слоя в Llama-7B) ├── Self-Attention │ ├── q_proj: [4096 × 4096] ← проекция в Query │ ├── k_proj: [4096 × 4096] ← проекция в Key │ ├── v_proj: [4096 × 4096] ← проекция в Value │ └── o_proj: [4096 × 4096] ← выходная проекция ├── MLP (Feed-Forward) │ ├── gate_proj: [4096 × 11008] │ ├── up_proj: [4096 × 11008] │ └── down_proj: [11008 × 4096] └── LayerNorm ``` Каждая из этих матриц — это просто **линейный слой**: $y = W_0 x + b$ (в Llama bias нет, но сути не меняет). **Вопрос:** при дообучении мы хотим, чтобы модель "перенастроила" эти проекции. В Full Fine-Tuning мы обновляем все $4096 \times 4096 = 16{,}777{,}216$ параметров каждой матрицы. В LoRA мы говорим: *"Эти изменения на самом деле лежат в низкоразмерном подпространстве, и нам не нужны 16 млн параметров для их описания"*. --- ## 2. Что добавляется: визуализация «до» и «после» ### ДО (оригинальный линейный слой): ``` Вход x [batch, seq_len, 4096] │ ▼ ┌──────────────┐ │ W_0 │ ← Заморожена (не обучается) │ [4096×4096] │ └──────────────┘ ▼ Выход y [batch, seq_len, 4096] ``` ### ПОСЛЕ добавления LoRA: ``` Вход x [batch, seq_len, 4096] │ ├──┐ ▼ ▼ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │ W_0 │ │ A │ ← Обучается │ [4096×4096] │ │ [4096 × r] │ (r = 8) │ ❄️ FROZEN │ └──────────────┘ └──────────────┘ └──────────────┘ │ │ │ ▼ ▼ ▼ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │ B │ ← Обучается │ [r × 4096] │ (r = 8) │ B·A [4096×4096] │ │ (но хранится как │ 4096×8 + 8×4096) │ └──────────────┘ └──────────────┘ └──────────────┘ ⊕──────────── Сумма ────────────⊕ │ ▼ Выход y [batch, seq_len, 4096] ``` **Формула:** $$y = W_0 x + \frac{\alpha}{r} \cdot B A x$$ Где: - $W_0 \in \mathbb{R}^{4096 \times 4096}$ — заморожена - $A \in \mathbb{R}^{4096 \times r}$ — обучается - $B \in \mathbb{R}^{r \times 4096}$ — обучается - $r = 8$ (типично) - $\alpha = 16$ (типично, $\alpha / r = 2$ — масштабный коэффициент) --- ## 3. Конкретный пример с числами ### Сколько параметров мы реально добавляем? Для **одной** матрицы `q_proj` в Llama-7B: | Компонент | Размерность | Параметры | |-----------|-------------|-----------| | Оригинальная W_0 | 4096×4096 | 16 777 216 | | LoRA матрица A | 4096×8 | 32 768 | | LoRA матрица B | 8×4096 | 32 768 | | **Всего добавлено** | | **65 536** | **Доля обучаемых параметров:** $\frac{65{,}536}{16{,}777{,}216} = 0.39\%$ Если мы вставляем LoRA только в `q_proj` и `v_proj` (классический рецепт из оригинальной статьи), то на каждый из 32 слоёв: ``` Обучаемые параметры на 1 слой = 2 × (4096×8 + 8×4096) = 131 072 Обучаемые параметры на 32 слоя = 32 × 131 072 = 4 194 304 ≈ 4.2M ``` Всего параметров модели: ~6.7B. Доля обучаемых: **~0.06%**. --- ## 4. Куда именно вставлять: `target_modules` Это ключевой практический вопрос. В коде `PEFT` это задаётся параметром `target_modules`: ### Вариант А: Классический (из оригинальной статьи LoRA) ```python LoraConfig(r=8, lora_alpha=16, target_modules=["q_proj", "v_proj"], # Только Query и Value lora_dropout=0.05,) ``` Обоснование: в оригинальной статье (Hu et al., 2021) показали, что **адаптация Query и Value** даёт почти такое же качество, как адаптация всех четырёх проекций, при вдвое меньшем количестве параметров. ### Вариант Б: Расширенный (современный стандарт для Llama) ```python LoraConfig(r=16, lora_alpha=32, target_modules=["q_proj", "k_proj", "v_proj", "o_proj", # Все 4 проекции Attention "gate_proj", "up_proj", "down_proj" # Все 3 линейных слоя MLP], lora_dropout=0.05,) ``` Обоснование: для моделей типа Llama, Mistral, Gemma практика показала, что **MLP-слои тоже важны** для адаптации, особенно при дообучении на специфичном домене. ### Визуализация: какие модули «патчатся» ``` Transformer Block ├── Self-Attention │ ├── q_proj: [4096×4096] ──→ +LoRA_A [4096×8] +LoRA_B [8×4096] ✅ │ ├── k_proj: [4096×4096] ──→ (frozen) ❌ [или +LoRA ✅] │ ├── v_proj: [4096×4096] ──→ +LoRA_A [4096×8] +LoRA_B [8×4096] ✅ │ └── o_proj: [4096×4096] ──→ (frozen) ❌ [или +LoRA ✅] ├── MLP │ ├── gate_proj: [4096×11008] ──→ (frozen) ❌ [или +LoRA ✅] │ ├── up_proj: [4096×11008] ──→ (frozen) ❌ [или +LoRA ✅] │ └── down_proj: [11008×4096] ──→ (frozen) ❌ [или +LoRA ✅] └── LayerNorm ──→ (всегда заморожен) ``` --- ## 5. Инициализация: почему модель не «ломается» в начале обучения Это критически важный момент. В начале обучения мы хотим, чтобы модель работала **точно так же**, как до добавления LoRA. ``` Инициализация: A ← Random Normal (Гауссовский шум, σ = 1/√r) B ← ZEROS (все нули) Результат при первом forward pass: ΔW = B × A = 0 × A = 0 y = W_0·x + (α/r)·0 = W_0·x ← Модель идентична оригиналу ``` По мере обучения B перестаёт быть нулевой и начинает "учиться" совместно с A. ### Почему именно такая асимметрия? Если бы обе матрицы были инициализированы нулями, градиенты не потекли бы (производная произведения двух нулевых матриц = 0). Если бы обе были случайными, модель сразу "сломалась" бы на первом же шаге. Асимметрия A=шум, B=0 решает обе проблемы. --- ## 6. Forward Pass: пошаговый расчёт Возьмём один токен из последовательности. Пусть входной вектор: $$x \in \mathbb{R}^{4096}$$ ### Шаг 1: Оригинальный путь $$y_{\text{orig}} = W_0 \cdot x \quad \text{(4096×4096) × (4096×1) = (4096×1)}$$ ### Шаг 2: Путь через LoRA $$h = A \cdot x \quad \text{(4096×8) × (4096×1) = (8×1)}$$ Заметьте: мы **сжали** 4096-мерный вектор до 8-мерного! Это и есть "бутылочное горлышко" (bottleneck), которое заставляет модель находить компактное представление изменений. $$\Delta y = B \cdot h \quad \text{(8×4096) × (8×1) = (4096×1)}$$ ### Шаг 3: Масштабирование и суммирование $$y = y_{\text{orig}} + \frac{\alpha}{r} \cdot \Delta y = W_0 x + \frac{16}{8} \cdot B A x = W_0 x + 2 \cdot B A x$$ ### Визуализация потока данных: ``` x [4096] │ ├──┐ │ ▼ ▼ W₀ [4096×4096] ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │ A [4096×8] │ │ │ Сжатие: 4096 → 8 │ "Что важно для задачи?" │ └──────────────┘ ▼ │ h [8] ← bottleneck │ ▼ │ ┌──────────────┐ │ └──────────────┘ │ B [8×4096] │ │ Расширение: 8 → 4096 │ "Как это применить?" │ └──────────────┘ ▼ │ Δy [4096] × (α/r) │ ▼ ▼ ⊕──────────── Сложение ────────────⊕ │ ▼ y [4096] ``` --- ## 7. Merging (слияние): что происходит при инференсе Это то, что делает LoRA уникальной среди всех PEFT-методов. После обучения мы можем **физически сложить***

*матрицы: $$W_{\text{merged}} = W_0 + \frac{\alpha}{r} \cdot B \cdot A$$ ``` Во время обучения: После слияния (инференс): ┌──────┐ ┌───┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ │ W₀ │ │ A │ │ B │ │ W₀ + α/r·BA │ │ 4096² │ │ │ │ → │ [4096²] │ └──────┘ └──────┘ └──────┘ └──────┘ └──────┘ 3 матрицы в памяти 1 матрица в памяти + ветвление в compute Нет ДОПОЛНИТЕЛЬНОЙ графа задержки (0 мс) ``` **Практическое следствие:** после слияния модель работает **с той же скоростью**, что и оригинал. Нет никаких дополнительных матричных умножений при генерации текста. Это главное преимущество LoRA перед, например, Adapter-методами, которые всегда добавляют задержку. --- ## 8. Что хранится на диске ### Полный файл модели (без LoRA): ``` llama-7b/ ├── model-00001-of-00002.safetensors (~13 GB) ├── model-00002-of-00002.safetensors (~13 GB) └── Итого: ~26 GB ``` ### LoRA-адаптер: ``` llama-7b-lora-medical/ ├── adapter_model.safetensors (~50-100 MB) ← только A и B матрицы ├── adapter_config.json (~1 KB) ← r, alpha, target_modules └── Итого: ~50-100 MB ``` **Разница:** 26 GB vs 50 MB. Вы можете хранить 500 адаптеров вместо одной копии модели. --- ## 9. Практический код: как это выглядит в PEFT ```python from peft import LoraConfig, get_peft_model from transformers import AutoModelForCausalLM # 1. Загружаем базовую модель model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b-hf") # 2. Конфигурируем LoRA lora_config = LoraConfig(r=8, # Ранг (размер "бутылочного горлышка") lora_alpha=16, # Масштабный коэффициент (α/r = 2) target_modules=[# Куда вставляем "q_proj", "v_proj", # Attention: Query и Value], lora_dropout=0.05, # Dropout для регуляризации bias="none", # Не трогаем биасы task_type="CAUSAL_LM",) # 3. "Патчим" модель model = get_peft_model(model, lora_config) # 4. Смотрим, сколько параметров обучается model.print_trainable_parameters() # Вывод: # trainable params: 4,194,304 || all params: 6,742,609,920 # trainable%: 0.0622% ``` --- ## 10. Интуиция: почему это работает Вернёмся к аналогии с автомобилем из Пункта 1: | Компонент | Аналогия | Что это в LoRA | |-----------|----------|----------------| | W_0 | Заводской двигатель | Замороженные pre-trained веса | | A (down-projection) | Датчик: "какие параметры важны для этой трассы?" | Сжимает 4096 → 8, выделяя ключевые признаки | | B (up-projection) | Привод: "как именно подкрутить каждый параметр" | Расширяет 8 → 4096, применяя изменения | | r | Количество "ручек настройки" | 8-64 степени свободы вместо 16 млн | | α / r | Чувствительность руля | Масштаб: насколько сильно адаптер влияет на выход | **Ключевая интуиция:** пространство полезных изменений весов для конкретной задачи имеет размерность $\ll 4096$. Ранг $r=8$ говорит: *"Мне достаточно 8 независимых направлений в 4096-мерном пространстве, чтобы описать все нужные изменения"*. И это действительно работает. --- *Хотите углубиться ещё в какой-то аспект (например, как ранг влияет на качество, или как работает QLoRA поверх LoRA), или переходим к Пункту 3: другие PEFT-методы (Adapters, Prefix Tuning, BitFit)?*

Response: Разберём устройство языковой модели и место LoRA без усложнённых терминов — от общего к частному.

1. Что представляет собой LLM

Большая языковая модель (LLM) — это **последовательный конвейер (слоёный пирог)** из абсолютно одинаковых блоков. В Llama-7B таких блоков **32 штуки**, они стоят друг за другом.

Когда вы отправляете текст в модель:

1. Каждый токен (слово или часть слова) превращается в вектор чисел (например, из 4096 элементов).
2. Этот вектор по очереди проходит сквозь **Слой 1** → **Слой 2** → **...** → **Слой 32**.
3. На выходе из 32-го слоя модель вычисляет следующее наиболее вероятное слово.

2. Где находятся Attention-слои и матрицы

Внутри **каждого** из 32 блоков находится два узла:

1. **Self-Attention (Внимание):** отвечает за контекст — помогает слову "понять", с какими другими словами в предложении оно связано (например, к чему относится местоимение «он»).
2. **MLP (Feed-Forward):** обрабатывает смысл самого слова и извлекает факты.

Где там математические матрицы?

Чтобы блок Self-Attention смог сопоставить слова между собой, входной вектор токена умножается на 3 основные матрицы (просто огромные таблицы с числами):

- W_q **(`q_proj`)** — Query (Запрос): "Что искало это слово?"
- W_k **(`k_proj`)** — Key (Ключ): "Какая тема у этого слова?"
- W_v **(`v_proj`)** — Value (Значение): "Какую информацию слово передаёт?"

Размер каждой такой таблицы в Llama-7B — 4096×4096 (около 16.7 млн чисел на одну матрицу).

3. Куда ИМЕННО вставляется LoRA

Вот главное заблуждение: **LoRA НЕ вставляется "между" слоями сети и НЕ меняет общую цепочку блоков.**

LoRA подключается **параллельно** существующей матрице (чаще всего к `q_proj` и `v_proj`). Это работает по принципу **байпаса (параллельного обводного канала)**:

1. Существующая матрица W_0 полностью **замораживается** (её 16.7 млн параметров перестают изменяться).
2. Рядом с ней строится **параллельный путь** из двух маленьких матриц A и B.

4. Как проходит сигнал через слой с LoRA

При дообучении вектор токена x (длиной 4096), доходя до линейной проекции (например, `q_proj`), **разделяется на два параллельных потока**:

$$\text{Выход } y = \underbrace{W_0 \cdot x}_{\text{Основной поток}} + \underbrace{B \cdot A \cdot x}_{\text{Поток LoRA}}$$

- **Основной поток ($W_0 \cdot x$):** проходит через зафиксированную фабричную матрицу. Это базовые знания модели, которые никогда не порпортит градиентный спуск.
- **Поток LoRA ($B \cdot A \cdot x$):**
  1. Вектор x умножается на тонкую матрицу A (размер 4096×8). Вектор резко сжимается от 4096 до 8 чисел.
  2. Полученные 8 чисел умножаются на матрицу B (размер 8×4096). Вектор обратно расширяется до 4096 чисел.
3. **Суммирование:** результаты двух путей складываются в один вектор y той же размерности (4096) и идут дальше по конвейеру блока Transformer.

Краткий итог

Вопрос	Ответ
Где физически находятся матрицы LoRA?	Внутри Transformer-блоков, прикреплённые **параллельно** к линейным проекциям Attention (`q_proj`, `v_proj`) и/или MLP.
Меняется ли архитектура сети?	Нет, длина конвейера не увеличивается. Размеры входящих и выходящих векторов остаются прежними.
Что происходит при инференсе?	После обучения матрицы $B \cdot A$ умножаются друг на друга и просто прибавляются к W_0. В итоговой модели остаётся стандартная структура без задержек в скорости.

User prompt: Ладно , давай начнём с начала :0)O NLP 1) Что такое RNN и зачем ему атеншен 2) Как работает трансформет для NLP 3) Что представляют из себя современные модели . Лучше начать с плана изучения , а потом мы пойдём по пунктам (я тут просто обнаружил , что частично всё это забыл и не мешало бы повторить)

Response: Ничего страшного, освежить фундамент — отличная идея. Эволюция NLP за последние 10 лет логична: каждое новое решение исправляло ключевой барьер предыдущего.

План повторения

- **Шаг 1: RNN и появление Attention** *(почему рекуррентным сетям стало тесно в фиксированном векторе)*
- **Шаг 2: Архитектура Transformer** *(как убрали рекурсию, добавили Self-Attention и распараллелили процесс)*
- **Шаг 3: Современные LLM** *(переход к Decoder-only, масштабирование, alignment и дообучение)*

Шаг 1: RNN и зачем ему Attention

1. Что такое RNN?

Рекуррентная нейросеть (RNN) была создана специально для обработки последовательностей (текст, звук, временные ряды). В отличие от обычных полносвязных сетей, у RNN есть **внутренняя память** (скрытое состояние h_t).

Сеть обрабатывает текст **пошагово, токен за токен**:

```
Токен x₁ ──► [ RNN Cell ] ──► h₁
                  │
Токен x₂ ──► [ RNN Cell ] ──► h₂
                  │
Токен x₃ ──► [ RNN Cell ] ──► h₃ (содержит информацию о x₁, x₂, x₃)
```

На каждом шаге t скрытое состояние пересчитывается по формуле:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Матрица весов W — одна и та же на всех шагах. Модель берёт предыдущий "контекст" h_{t-1}, добавляет новое слово x_t и получает обновлённый контекст h_t.

2. Главные проблемы классических RNN (LSTM / GRU)

1. **Последовательная обработка (невозможность параллелизма):** Чтобы вычислить h_{100}, вы **обязаны** последовательно посчитать $h_1 \dots h_{99}$. Из-за этого RNN нельзя эффективно обучать на тысячах видеокарт одновременно.
2. **Бутылочное горлышко (Information Bottleneck):** В задачах машинного перевода или суммаризации использовалась схема **Encoder-Decoder**:
   - **Encoder** вычитывал входной текст (например, из 50 слов) и сжимал его в **последний единственный вектор** h_T.
   - **Decoder** пытался сгенерировать ответ, опираясь *только* на этот единственный вектор h_T.

   Представьте, что вас попросили прочитать роман на 300 страниц, записать его краткую суть на одной карточке, а затем по этой карточке перевести весь роман слово в слово. На длинных текстах модель неизбежно забывала начало предложения.

3. Зачем появился механизм Attention (Внимание)?

В 2014 году (Бахданау и др.) предложили идею: **зачем заставлять модель сжимать всю историю в один вектор?**

Давайте сохранять скрытые состояния $h_1, h_2, \dots, h_T$ для **каждого** слова входной последовательности, а декодеру дадим возможность "смотреть" на весь список этих состояний и выбирать самые важные.

Как работает Attention в RNN:

1. На каждом шаге генерации декодер сравнивает своё текущее состояние с **каждым** промежуточным вектором этopic/слова из энкодера.
2. Вычисляются **оценки близости (scores)**, которые через $\mathrm{Softmax}$ превращаются в вероятности — **веса внимания (** $\alpha_1, \alpha_2, \dots, \alpha_T$**)**.
3. Декодер берет взвешенную сумму всех векторов входа:

$$c = \sum_{i=1}^{T} \alpha_i h_i$$

Если декодер переводит слово «кошка», он получит максимальный вес α_i напротив исходного слова «cat», прямо "сфокусировав взгляд" на нем, проигнорировав остальные слова.

Что изменилось после этого?

Attention полностью решил проблему "забывания" длинных контекстов в RNN. Однако оставался главный барьер: **последовательный цикл $t_1 \to t_2 \to t_3$ всё ещё не позволял распараллелить обучение**.

В 2017 году авторы статьи *"Attention Is All You Need"* задали вопрос: *«А что если выкинуть RNN вообще и оставить ТОЛЬКО Attention?»* — так родился **Transformer**.

User prompt: Можешь подробнее расписать про то , что такое матрицы $$h_t = \text{tanh}(W_{hh} h_{t-1} + W_{xh} x_t)$$, что значат индексы и т.д

Response: Формула $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ — это математический механизм обновления памяти RNN-ячейки на каждом шаге.

1. Расшифровка индексов и переменных

Векторы (сигналы на шаге t):

- t — временной шаг (индекс токена в последовательности). Например: $t = 1$ для первого слова, $t = 2$ для второго.
- x_t — вектор текущего входящего слова (эмбеддинг токена на шаге t).
- h_{t-1} — вектор памяти (скрытого состояния) с **предыдущего** шага. Хранит всю информацию, которую сеть накопила от начала текста до слова $t-1$.
- h_t — вектор памяти на **текущем** шаге. Это обновлённый вектор контекста, который пойдёт дальше к слову $t+1$.

Матрицы весов (обучаемые параметры):

Индексы матриц обозначают **«откуда $\to$ куда»** передаётся сигнал:

- W_{xh} ($x \to h$, input-to-hidden) — матрица, которая отвечает за **перевод нового входящего слова** x_t в пространственное представление скрытого состояния h.
- W_{hh} ($h \to h$, hidden-to-hidden) — матрица, которая отвечает за **трансформацию и удержание предыдущей памяти** h_{t-1}.

2. Разбор с конкретными размерностями

Допустим, мы обрабатываем предложение. Зададим конкретные размеры векторов:

- Размер эмбеддинга слова (d_x) = **300** (вектор x_t состоит из 300 чисел).
- Размер скрытого состояния памяти (d_h) = **512** (вектор h_t состоит из 512 чисел).

Посмотрим, как матрицы приводят всё к единому размеру $d_h = 512$:

$$h_t = \tanh\left(\underset{[512\times512]}{W_{hh}} \cdot \underset{[512\times1]}{h_{t-1}} + \underset{[512\times300]}{W_{xh}} \cdot \underset{[300\times1]}{x_t} \right)$$

Шаг за шагом:

1. **Преобразование входа:** $W_{xh} \cdot x_t \implies [512 \times 300] \times [300 \times 1] = [512 \times 1]$ Матрица W_{xh} переводит 300-мерный вектор слова в 512-мерный вектор "впечатления" от этого слова.
2. **Преобразование памяти:** $W_{hh} \cdot h_{t-1} \implies [512 \times 512] \times [512 \times 1] = [512 \times 1]$ Матрица W_{hh} перерабатывает 512-мерный вектор старой памяти, чтобы решить, что из прошлых данных актуально.
3. **Сложение:** оба получившихся вектора имеют размерность $[512 \times 1]$. Мы складываем их элемент за элементом.
4. **Активация $\tanh$:** зажимает каждое из 512 чисел получившегося вектора в диапазон от -1 до $+1$. Это предотвращает бесконечный рост чисел при последовательной обработке сотен слов.

   Примечание: В полных реализациях к этой сумме также добавляют вектор смещения (bias) b_h размерности $[512 \times$

$1]$, но суть операции от этого не меняется.

3. Интуитивный смысл операций

Если перевести формулу на человеческий язык:

$$\text{Новая память } h_t = \text{Сжать_до_(-1..1)}\Big(\text{(Как осмыслить прошлое)} \cdot h_{t-1} + \text{(Как осмыслить новое слово)} \cdot x_t\Big)$$

- **Матрицы W_{hh} и W_{xh} одинаковы на каждом шаге t.** В процессе обучения с помощью градиентного спуска сеть настраивает эти две таблицы весов так, чтобы:
  - W_{xh} выделяла в новом слове важную информацию.
  - W_{hh} решала, какую часть старой информации объединить с новым словом, а какую — сгладить.

User prompt: Отлично, с математикой RNN разобрались. Давай перейдём к Шагу 2: как работает Transformer, Self-Attention, матрицы K, Q, V и почему он заменил RNN.

Response: Переход от RNN к Transformer в 2017 году стал главным прорывом в NLP. Модель Transformer отказалась от рекурсии, заменив её быстрым и полностью параллельным механизмом **Self-Attention** (самовнимания).

1. Почему Transformer заменил RNN?

Главным ограничением RNN была **последовательная природа**: чтобы обработать 100-е слово, нужно было последовательно прогнать через ячейку предыдущие 99 слов. Это создавало вычислительное "бутылочное горлышко".

Критерий	RNN (LSTM / GRU)	Transformer
Принцип работы	Последовательный цикл (слово за словом)	Параллельная обработка всей матрицей
Обучение на GPU	Плохо масштабируется (потоки ждут шаг $t-1$)	Идеально распараллеливается на тысячи ядер
Дальние связи	Забывает начало текста на длинных дистанциях	Связывает любые два слова за $O(1)$ операций
Скорость обучения	Медленная	В десятки раз быстрее при тех же объёмах

2. Что такое Self-Attention (Самовнимание)?

Если в RNN механизм внимания связывал *декодер с энкодером*, то в **Self-Attention** последовательность смотрит **сама на себя**.

Каждое слово в предложении сравнивается со всеми остальными словами этого же предложения, чтобы понять их взаимосвязь.

> **Пример:** *«Заяц не перебежал дорогу, потому что **он** был утомлён».*
>
> Чтобы понять, к чему относится слово «**он**» (к зайцу или к дороге), Self-Attention вычисляет степень связи между «он» и всеми предыдущими словами. Связь с «зайцем» получит максимальный вес внимания.

3. Матрицы Q, K, V: Аналогия и математика

Для реализации Self-Attention входные векторы токенов раскладываются на три роли, аналогичные работе поисковой системы или базы данных:

- **Query (Q) — Запрос:** «Что это слово ищет в контексте?»
- **Key (K) — Ключ:** «Какую тему/характеристику описывает это слово?»
- **Value (V) — Значение:** «Какое реальное смысловое содержание несет слово, если оно подошло?»

Как создаются Q, K, V из входных данных?

Входная матрица токенов X (размерности $[\text{seq_len} \times d_{\text{model}}]$) умножается на три обучаемые матрицы весов W_Q, W_K, W_V:

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V$$

Формула Scaled Dot-Product Attention

Вся магия самовнимания описывается одной формулой:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{d_k}\right)V$$

Разберём её по шагам:

1. $Q \cdot K^T$ **(Перемножение запросов и ключей):** Скалярное произведение вектора Q одного слова на вектор K другого показывает, насколько они «созвучны». Результат — матрица скалярных оценок размером $[\text{seq_len} \times \text{seq_len}]$, где какова ячейка (i, j) отвечает за релевантность слова i слову j.
2. **Деление на** d_k **(Масштабирование):** d_k — это размерность векторов ключей (например, 64). При больших размерностях скалярные произведения становятся очень крупными, из-за чего функция Softmax выдаёт значения, близкие к 0 или 1 (крайние области с практически нулевыми градиентами). Деление на d_k стабилизирует обученин.
3. $\text{Softmax}(\dots)$ **(Нормализация):** Переводит оценки близости в вероятности от 0 до 1. Сумма весов в каждой строке становится равной 1.0.
4. **Умножение на** V **(Взвешенная сумма):** Полученные вероятности используются как коэффициенты для суммирования векторов значений V. Слово получает обновлённый вектор, обогащённый контекстом всех релевантных соседей.

4. Две важные детали: Positional Encoding и Multi-Head

1. Позиционное кодирование (Positional Encoding)

Поскольку Transformer обрабатывает все слова одновременно, для него предложения *«Кот съел мышь»* и *«Мышь съела кота»* без дополнительных данных выглядели бы идентично.

Чтобы передать порядок слов, к эмбеддингу каждого токена прибавляют вектор **положения** (синусоидальные сигналы в оригинале или ротационные эмбеддинги **RoPE** в современных LLM).

2. Многоголовое внимание (Multi-Head Attention)

Вместо одного расчета Q, K, V модель делает их параллельно в нескольких "головах" (например, 8, 16 или 32 головы).

- Одна голова учится отслеживать грамматические связи (глагол + существительное).
- Вторая — смысловые синонимы.
- Третья — ссылки местоимений.

Результаты всех голов объединяются (concatenation) и проецируются через отдельную линейную матрицу W_O.

User prompt: Теперь перейдём к тому , как устроены современные модели и куда выткают LoRank методами соотв матрицы

Response: Современные языковые модели (Llama, Mistral, Qwen, DeepSeek) — это развитые вариации архитектуры **Decoder-Only Transformer**.

Чтобы понять, куда именно подключается LoRA, разберём сначала, из чего состоит один блок современной LLM, а затем посмотрим на точки интеграции матриц адаптации.

1. Как устроена современная LLM (Decoder-Only)

В отличие от оригинального Трансформера 2017 года (где был энкодер и декодер), современные модели используют **только**

декодер. Модель генерирует текст авторегрессионно: каждый следующий токен предсказывается только на основе предыдущих.

Каждый из 32–80 блоков современной LLM содержит **два ключевых узла** и 4 главных технологических улучшения:

```
Transformer Block (повторяется N раз, например 32 в Llama-7B)
├── 1. RMSNorm (быстрая нормализация входа)
├── 2. Grouped-Query Attention (GQA)
│     ├── RoPE (ротационные эмбеддинги для позиций)
│     ├── q_proj  [4096 × 4096]  ← Проекция Query
│     ├── k_proj  [4096 × 1024]  ← Проекция Key (сжата в GQA)
│     ├── v_proj  [4096 × 1024]  ← Проекция Value (сжата в GQA)
│     └── o_proj  [4096 × 4096]  ← Выходная проекция (Output)
├── 3. RMSNorm (вторая нормализация)
└── 4. MLP / Feed-Forward (SwiGLU)
      ├── gate_proj [4096 × 11008] ← Матрица гейтинга
      ├── up_proj   [4096 × 11008] ← Входная проекция
      └── down_proj [11008 × 4096] ← Выходная проекция
```

Ключевые модернизации современного блока:

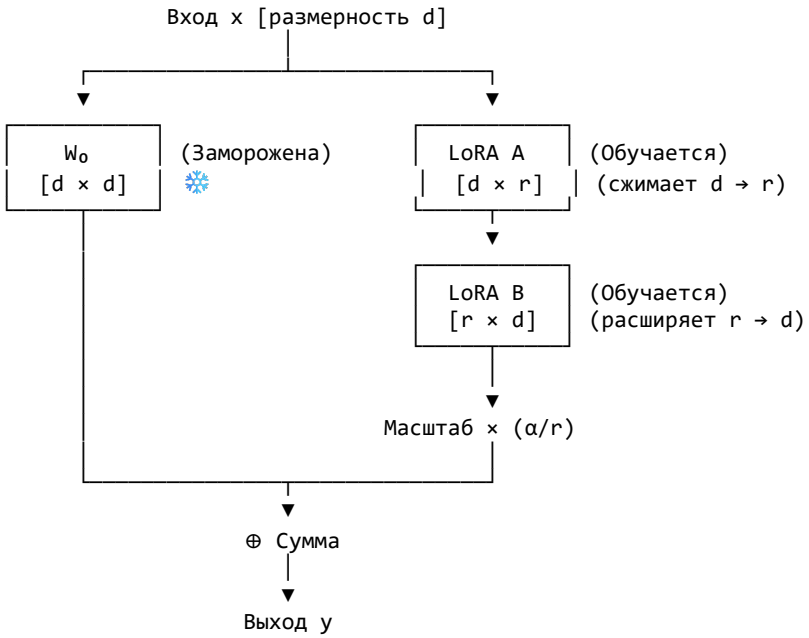
1. **GQA (Grouped-Query Attention):** Головы Query делятся на группы, которые делят между собой общие головы Key и Value. Это уменьшает объём памяти под KV-кэш во время генерации.
2. **SwiGLU в MLP:** Вместо 2 матриц в обычном Feed-Forward слое используются 3 матрицы (`gate_proj`, `up_proj`, `down_proj`). Этот узел отвечает за хранение и извлечение фактов.
3. **RMSNorm:** Упрощённая и более быстрая версия LayerNorm.
4. **RoPE (Rotary Position Embedding):** Позиционная информация внедряется напрямую в векторы Q и K с помощью поворота векторов в комплексноподобном пространстве.

2. Куда и как «втыкаются» матрицы LoRA

LoRA (Low-Rank Adaptation) не добавляет новые слои между блоками и не меняет глубину сети. Она подключается **параллельно** к существующим **линейным проекциям** (умножениям матриц) внутри блоков.

Каждая линейная проекция в PyTorch — это операция $y = W_0 \cdot x$.

LoRA оборачивает эту проекцию в параллельный «байпас»:



Формула вычисления:

$$y = W_0 \cdot x + \frac{\alpha}{r} \cdot (B \cdot A \cdot x)$$

Где:

- r **(Ранг):** маленькое число (например, $r = 8$ или $r = 16$).
- **Матрица** A **(**$d \times r$**):** сжимает входной вектор высокой размерности d до узкого горлышка r.
- **Матрица** B **(**$r \times d$**):** восстанавливает вектор обратно из r до исходной размерности d.

3. Какие именно модули обновляют через LoRA?

В параметрах библиотеки PEFT это задаётся переменной `target_modules`:

Подход	К каким проекциям подключаем	Почему так делают
Классический (оригинальная статья LoRA)	`["q_proj", "v_proj"]`	Минимальное число параметров (всего ~0.06%). Достаточно для изменения стиля или адаптации к простым инструкциям.
Современный стандарт (Llama / Qwen)	`["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]`	Подключается **ко всем 7 линейным матрицам** каждого блока. Задействует MLP-слои, где хранятся знания, что критично для освоения новых узких доменов (медицина, юриспруденция, программирование).

4. Что происходит при сохранении и инференсе

1. **При сохранении:** На диск записываются **только** маленькие матрицы A и B для каждого целевого слоя. Для Llama-7B весь адаптер занимает всего **50–100 МБ** вместо 13–26 ГБ базовой модели.
2. **При инференсе (Слияние весов):** Чтобы LoRA не замедляла генерацию дополнительными умножениями, матрицы перемножают и физически прибавляют к базовым весам:

$$W_{\text{final}} = W_0 + \frac{\alpha}{r} \cdot (B \cdot A)$$

После этого слияния модель снова становится стандартной LLM с 0 мс дополнительной задержки.

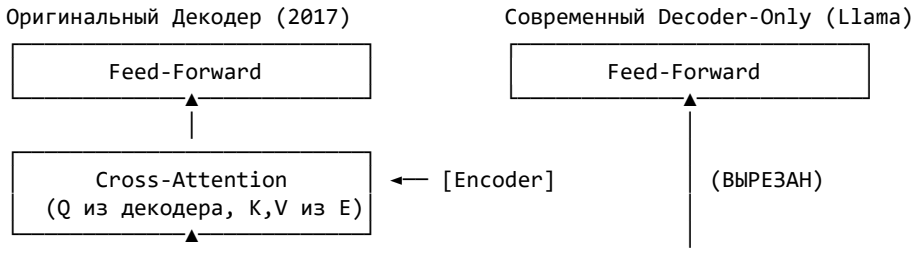
User prompt: Но ведь в декодере 2 атеншен , один из которых берёт результат энкодера , которого нет

Response: Вы подметили важнейшую деталь! В оригинальном трансформере 2017 года (Seq2Seq) в декодере действительно было **два** слоя внимание:

1. **Masked Self-Attention:** смотрел на предыдущие сгенерированные токены самого декодера.
2. **Cross-Attention (Encoder-Decoder Attention):** брал Запросы (Q) из декодера, а Ключи (K) и Значения (V) — из выходов энкодера.

В современных архитектурах **Decoder-Only** (Llama, GPT-4, Qwen, DeepSeek) **второй слой (Cross-Attention) просто полностью удалили**.

Сравнение блоков декодера





Как модель работает без Cross-Attention?

В оригинальном Seq2Seq вводные данные и ответ были **разделены**:

- Энкодер считывал вход: *«Переведи: Hello world»*
- Декодер генерировал выход: *«Привет мир»*, подглядывая в энкодер через Cross-Attention.

В современной архитектуре **Decoder-Only** промпт и ответ объединены в **одну единую последовательность**:

$$\text{Токены: Переведи: Hello world} \quad \underbrace{\text{Привет}}_{\text{Сгенерировано}} \quad \underbrace{\text{мир}}_{\text{Следующий токен}}$$

Промпт

Единственный оставшийся слой внимания — **Causal Self-Attention** — делает всю работу сразу:

1. Когда модель обрабатывает токены **промпта**, токены смотрят друг на друга и формируют понимание инструкции.
2. Когда модель генерирует **ответ**, новый токен через этот же Self-Attention смотрит **и на предыдущие слова ответа, и назад на весь промпт**.

Таким образом, отпала необходимость в отдельном механизме связи — всё общение между промптом и ответом происходит внутри **одного** слоя самовнимания.

User prompt: А чем отличаются матрица запросов, ключей и значений, почему у них есть симантическая привязка

Response: Чтобы понять, чем отличаются Q, K, V и откуда у них берётся «семантический смысл», нужно посмотреть на две вещи: **математическую необходимость** (почему нельзя обойтись одной матрицей) и **механику обучения** (как случайные числа обретают смысл).

1. Функциональное различие матриц

Из одного и того же входящего вектора слова x (например, размерностью 4096) мы получаем три абсолютно разных вектора с помощью умножения на матрицы весов W_Q, W_K, W_V:

Вектор	Формула	Роль в системе	Разговорный аналог
Query (q)	$q = W_Q \cdot x$	**Потребность:** Что этому слову нужно узнать от других слов?	*«Мне нужно существительное женского рода, к которому я принадлежу»*
Key (k)	$k = W_K \cdot x$	**Идентификатор:** Какими свойствами обладает это слово для поиска?	*«Я — существительное женского рода, кошка»*
Value (v)	$v = W_V \cdot x$	**Полезная нагрузка:** Какую инфомацию слово передаст дальше, если его выбрали?	*«Передай в следующий слой: [животное, пушистый, подлежащее]»*

2. Математическая необходимость: почему Q, K и V разделены?

Если бы мы использовали исходный вектор x напрямую без разделения на Q, K, V, система внимания бы сломалась. Разделение нужно по двум фундаментальным причинам:

Причина 1: Асимметрия отношений ($Q \neq K$)

Внимание между словами **несимметрично**:

- В предложении *«Кот увидел мышь, потому что **она** убегала»* местоимение «**она**» критически нуждается в слове «**мышь**», чтобы понять свой смысл.

- Но слову «**мышь**» само по себе местоимение «**она**» для понимания собственного значения не нужно.

Если бы мы считали внимание напрямую между векторами входа $X \cdot X^T$, матрица связей была бы строго симметричной ($A_{i,j} = A_{j,i}$).

Умножение $Q \cdot K^T$ создаёт **направленный граф**:

$$q_{\text{она}} \cdot k_{\text{мышь}}^T \neq q_{\text{мышь}} \cdot k_{\text{она}}^T$$

Причина 2: Разделение «условия совпадения» и «содержимого» ($K \neq V$)

- **Key (K)** отвечает только за **поиск** (matching score). Его задача — совпасть с Q других слов через скалярное произведение.
- **Value (V)** отвечает за **транспортировку смысла** в следующие слои.

Если объединить K и V в один вектор, возникнет конфликт задач: вектор должен будет одновременно и идеально подходить под поисковые фильтры соседних слов, и нести в себе богатство семантического описания. Разделение K и V развязывает модели руки.

3. Откуда берётся «семантическая привязка»?

Изначально (при инициализации модели) матрицы W_Q, W_K, W_V заполнены **абсолютно случайными числами**. В этот момент у них нет никакой семантики.

Семантическая привязка — это **эмерджентное свойство**, которое появляется исключительно под воздействием **обратного распространения ошибки (Backpropagation)** при решении итоговой задачи (например, предсказания следующего слова).

`1. Случайные веса ⟶ 2. Неверное предсказание ⟶ 3. Ошибка (Loss) ⟶ 4. Градиент подправляет W_Q, W_K, W_V`

Как градиент наводит порядок в пространствах Q, K, V:

1. **Формирование пространства Q и K:** Чтобы сгенерировать правильное следующее слово, модели критически важно связать прилагательное с его существительным. Градиентный спуск начинает поворачивать строки матрицы W_Q и W_K так, чтобы:
    - Векторы Q для слов-запросов (местоимений, глаголов) в многомерном пространстве **поворачивались в ту же сторону**, где лежат векторы K для соответствующих им контекстных слов (существительных).
    - Скалярное произведение $q \cdot k$ становилось максимальным там, где передача информации приносит пользу для уменьшения ошибки.
2. **Формирование пространства V:** Матрица W_V под воздействием градиента учится проецировать исходный вектор токена в такое подпространство, где отсекается лишний шум, а важные семантические признаки (число, род, время, предметная область) раскладываются по удобным для следующих слоёв координатам.

Итог

Семантика Q, K, V — это не зашитое программистами правило, а **оптимальная геометрия многомерных векторов**, к которой модель вынужденно приходит, чтобы минимизировать ошибку предсказания текста.

User prompt: Можешь добавить к этому объяснению loss функцию внимания т.к она непосредственно влияет на формирование семантики и специализации матриц , объясни как

Response: В самой архитектуре Transformer **нет отдельной изолированной лосс-функции внутри блока Attention**.

Формирование семантики и геометрии матриц W_Q, W_K, W_V происходят под действием **общей функции потерь модели** — кросс-энтропии (Cross-Entropy Loss) на выходе сети при градиентном прогоне (Backpropagation).

1. Главная лосс-функция сети

Для авторегрессионной языковой модели лосс-функция на одном шаге генерации предсказываемого токена y_t выглядит как стандартная **Cross-Entropy**:

$$\mathcal{L} = -\log P(y_t \mid x_{<t}) = -\log\left(\frac{e^{z_{y_t}}}{\sum_{k=1}^{V} e^{z_k}}\right)$$

где:

- z — вектор логитов размерности словаря V на самом верхнем слое модели (после `LM Head`).
- y_t — индекс правильного следующего токена.

Модель хочет максимизировать вероятность правильного токена y_t, то есть минимизировать $\mathcal{L}$.

2. Граф прохождения градиента до матриц W_Q, W_K, W_V

Чтобы понять, как $\mathcal{L}$ заставляет матрицы W_Q, W_K, W_V специализироваться, проследим путь градиента назад по цепному правилу (Chain Rule):

$$\mathcal{L} \longrightarrow z\ (\text{Logits}) \longrightarrow Y\ (\text{Выход слоя}) \longrightarrow \underbrace{S \cdot V}_{\text{Attention Out}} \longrightarrow S = \underbrace{\text{softmax}(A)}_{\text{Веса внимания}} \longrightarrow A = \underbrace{\frac{QK^T}{d_k}}_{\text{Логиты внимания}} \longrightarrow \begin{cases} Q = XW_Q \\ K = XW_K \\ V = XW_V \end{cases}$$

Каждый узел этой цепи накладывает свои математические условия на то, **как именно** должны измениться матрицы.

3. Математика дифференцирования: как градиент форсует роли

А. Матрица Значений W_V (Формирование содержимого)

Выход слоя внимания для последовательности: $O = S \cdot V$, где $V = XW_V$, а S — матрица весов внимания $[N \times N]$.

Градиент по матрице W_V равен:

$$\frac{\partial \mathcal{L}}{\partial W_V} = X^T \cdot \left(S^T \cdot \frac{\partial \mathcal{L}}{\partial O}\right)$$

- **Как это формирует семантику W_V:** Градиент $\frac{\partial \mathcal{L}}{\partial O}$ указывает, каких признаков не хватило на выходе, чтобы верно угадать токен y_t. Умножение на S^T взвешивает этот сигнал с учётом того, какие токены смоделировали связь. В результате W_V обновляется так, чтобы извлекать из входящего вектора X_i только **полезную семантическую нагрузку** (род, число, фактические свойства), отсекая шум.

Б. Матрицы Запросов W_Q и Ключей W_K (Геометрия поиска)

Градиент сначала проходит через дифференцирование функции Softmax к скалярным произведениям (логитам внимания) $A_{ij} = \frac{q_i \cdot k_j^T}{d_k}$:

$$\frac{\partial \mathcal{L}}{\partial A} = \left(\frac{\partial \mathcal{L}}{\partial O} \cdot V^T\right) \odot \text{Softmax}'(A)$$

Затем частные производные по W_Q и W_K принимают вид:

$$\frac{\partial \mathcal{L}}{\partial W_Q} = \frac{1}{\sqrt{d_k}} X^T \cdot \left(\frac{\partial \mathcal{L}}{\partial A} \cdot K\right)$$

$$\frac{\partial \mathcal{L}}{\partial W_K} = \frac{1}{d_k} X^T \cdot \left(\left(\frac{\partial \mathcal{L}}{\partial A}\right)^T \cdot Q\right)$$

4. Геометрическая интуиция: поворот векторов при обучении

Посмотрим на шаг градиентного спуска для пары векторов q_i (запрос i-го слова) и k_j (ключ j-го слова):

$$W_Q \leftarrow W_Q - \eta \frac{\partial \mathcal{L}}{\partial W_Q}, \quad W_K \leftarrow W_K - \eta \frac{\partial \mathcal{L}}{\partial W_K}$$

1. **Если связь между токенами i и j необходима** для снижения общей ошибки (градиент $\frac{\partial \mathcal{L}}{\partial A_{ij}} < 0$):
   - Градиентный шаг **прибавляет** вектор k_j к направлению q_i, а вектор q_i — к направлению k_j.
   - Векторы $q_i = X_i W_Q$ и $k_j = X_j W_K$ **поворачиваются навстречу друг другу** в d_k-мерном пространстве. Их косинусное сходство растёт, а скалярное произведение $q_i \cdot k_j^T$ увеличивается.
2. **Если связь ложная или мешает предсказанию** ($\frac{\partial \mathcal{L}}{\partial A_{ij}} > 0$):
   - Градиент **разворачивает** векторы q_i и k_j в противоположные стороны (вплоть до ортогональности), обнуляя вес внимания между ними.

```
До оптимизации (случайные W_Q, W_K):        После обратного прохода (Backprop):

        q_i                                         q_i
         /                                           /
        /  θ (большой угол)                         / θ → 0° (векторы совпали)
       ------------> k_j                           ------------> k_j
q_i · k_j^T ≈ 0                             q_i · k_j^T >> 0
```

5. Почему происходит разделение по «головам» (Multi-Head)

При наличии H голов внимание вычисляется параллельно:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W_O$$

- Каждая голова h имеет свои случайные начальные матрицы $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$.
- Поскольку градиент $\frac{\partial \mathcal{L}}{\partial \text{head}_h}$ распределяется через объединение (Concat) и матрицу W_O, **головам энергетически выгоднее специализироваться на разных подзадачах**.
- Если Голова 1 уменьшила ошибку за счёт отслеживания синтаксических связей, её градиенты уводят веса $W_Q^{(1)}, W_K^{(1)}$ в одно подпространство. У Головы 2 в это время остаётся стимул закрывать другие типы связей (семантические синонимы, косвенные ссылки), поворачивая свои векторы в ортогональные подпространства.

User prompt: Давай теперь на примере микронейронки , состоящей из 3х блоков , посмотрим , как она обучается и что обучается , как она работает (мы всё время говорили про токены , но ведь после прохождения одного блока это уже не токены а что-то другое)

Response: Вы абсолютно правы в своём замечании: **после прохождения даже одного блока это уже давно не «токены»**.

Давайте разберёмся на примере **микро-модели**, чтобы увидеть этот процесс физически — с размерностями, векторами и тем, что именно меняется при обучении.

1. Наша «микро-модель» для теста

Представим упрощённую модель:

- **Размер словаря (V):** всего 1000 слов (токенов).
- **Длина последовательности:** 2 токена («Кот», «спит»).
- **Размер скрытого состояния (размерность векторов d_{model}):** всего **4** (вместо 4096 в Llama).
- **Количество блоков Transformer:** ровно **3** (вместо 32).

2. Эволюция данных: от токенов к скрытым состояниям

Шаг 0: Вход (Токены)

Мы подаем в модель не слова, а числовые ID из словаря:

- Токен №42 = «Кот»
- Токен №105 = «спит»

Шаг 1: Эмбеддинг (Превращение в векторы)

Слой `Embedding` берёт эти ID и подставляет для каждого слово-вектор размерности 4:

- Вектор токена «Кот»: $X_0^{(1)} = [0.1,\ -0.5,\ 0.8,\ 0.2]$
- Вектор токена «спит»: $X_0^{(2)} = [0.3,\ 0.1,\ -0.2,\ 0.9]$

На выходе получается матрица входа размерности $[2 \times 4]$ (2 токена, в каждом по 4 признака). **Это ещё не контекст, это просто заученные статические смыслы слов.**

3. Пошаговый Forward Pass через 3 блока

Теперь эта матрица $[2 \times 4]$ отправляется в конвейер из трёх блоков. Внутри каждого блока происходят две вещи: **Self-Attention** (смешивание смыслов между словами) и **MLP** (обработка каждого слова по отдельности).

⚙️ Блок 1 (Низкоуровневые связи)

- Векторы X_0 заходят в Блок 1.
- Срабатывает Attention: слово «спит» смотрит на слово «Кот» и понимает, кто именно совершает действие.
- Векторы меняют свои значения. Теперь они **контекстуализированы**.
- **Результат на выходе Блока 1 (X_1):**
  - $X_1^{(1)} = [0.2,\ -0.3,\ 0.7,\ 0.4]$ *(теперь «Кот» несёт оттенок субъекта)*
  - $X_1^{(2)} = [0.4,\ 0.2,\ -0.1,\ 0.8]$ *(теперь «спит» знает, что спит именно кот)*

⚙️ Блок 2 (Промежуточная логика)

- Матрица X_1 $[2 \times 4]$ заходит во **второй блок**.
- Attention и MLP второго блока уточняют связи, добавляя грамматические и синтаксические свойства.
- **Результат на выходе Блока 2 (X_2):** векторы снова изменились, стали более абстрактными.

⚙️ Блок 3 (Высокоуровневые абстракции)

- Матрица X_2 заходит в **третий (последний) блок**.
- Здесь формируется итоговое смысловое представление всей фразы.
- **Результат на выходе Блока 3 (X_3):** финальная матрица $[2 \times 4]$, где последний вектор хранит сжатый смысл всей предыстории предложения.

4. Финал: Предсказание следующего слова

Финальный вектор второго токена после Блока 3 (размерностью 4) отправляется в выходной слой (`LM Head`).

`LM Head` — это обычная матрица весов размером $[4 \times 1000]$ (где 1000 — размер нашего словаря). Она умножает наш вектор $[1 \times 4]$ на матрицу $[4 \times 1000]$ и получает вектор-строку из 1000 чисел (**логитов**):

$$\text{Логиты} = \text{Вектор}_{[\mathbf{1} \times \mathbf{4}]} \times \text{Матрица}_{[\mathbf{4} \times \mathbf{1000}]} = \text{Вектор}_{[\mathbf{1} \times \mathbf{1000}]}`$$

Каждое из 1000 чисел соответствует вероятности определенного слова из словаря стать следующим. Функция `Softmax` превращает их в проценты. Модель выдает, например: *«Следующее слово с вероятностью 85% — „на“»*.

5. Как это всё обучается и ЧТО именно меняется?

Допустим, модель ошиблась и вместо «на диване» предсказала слово «громко». Возникает ошибка (**Loss**).

Если это Full Fine-Tuning (полное дообучение):

Графический поток ошибки идёт задом наперёд (Backpropagation) через все три блока:

1. Ошибка меняет веса в выходном слое (`LM Head`).
2. Проходит сквозь **Блок 3** $\rightarrow$ меняются все матрицы внутри Блока 3 (`q_proj`, `k_proj`, `v_proj`, `gate_proj`, `up_proj`, `down_proj`).
3. Проходит сквозь **Блок 2** $\rightarrow$ меняются веса Блока 2.
4. Проходит сквозь **Блок 1** $\rightarrow$ меняются веса Блока 1.

Итог: Пересчитываются абсолютно все параметры во всех трёх блоках. Модель может «забыть» старые знания (катастрофическое забывание).

Если мы используем LoRA:

1. Базовые матрицы W_0 внутри Блоков 1, 2 и 3 **намертво заморожены** (они не меняются ни на один бит).
2. Рядом с ними во всех трёх блоках прикреплены маленькие матрицы A и B (наши адаптеры).
3. Ошибка проходит сквозь сеть, но **градиенты оседают только в матрицах A и B всех трех блоков**.
4. В итоге обновляются только эти мини-матрицы, а исходная языковая база модели остаётся нетронутой.

User prompt: Покажи на конкретных маленьких матрицах (например, 2x2), как делается один шаг градиентного спуска для LoRA-матриц A и B.

Response: Разберём работу LoRA нагляднейшим образом — на крошечных матрицах размером 2×2. Мы проведем полный цикл: прямой проход (Forward), подсчёт ошибки, обратный проход (Backward) и один шаг градиентного спуска.

1. Исходные данные нашей микро-модели

1. **Входной вектор (x):** Пусть к нам пришёл вектор размерности 2×1:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

2. **Замороженная базовая матрица (W_0):** Размер 2×2:

$$W_0 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

3. **LoRA-адаптер (Ранг $r = 1$):**
   - Поскольку ранг $r = 1$, матрица сжатия A имеет размерность $[1 \times 2]$, а матрица восстановления B — размерность $[2 \times 1]$.
   - **Инициализация:** по стандартам LoRA матрица A заполнена случайными числами, а матрица B заполнена **нулями** (чтобы в самом начале $\Delta W = 0$ и модель работала идентично оригиналу).
   - Пусть:

$$A = \begin{bmatrix} 0.1 & 0.2 \end{bmatrix}, \quad B = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

4. **Целевой ответ (Target):** Допустим, мы хотим, чтобы модель на выходе выдала вектор:

$$y_{\text{target}} = \begin{bmatrix} 6 \\ 10 \end{bmatrix}$$

5. **Скорость обучения (Learning Rate, η):** $\eta = 0.1$.

Шаг 1: Прямой проход (Forward Pass)

Считаем результат работы слоя: $y = W_0 \cdot x + B \cdot A \cdot x$.

1. **Основной поток ($W_0 \cdot x$):**

$$y_{\text{orig}} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1(1) + 2(2) \\ 3(1) + 4(2) \end{bmatrix} = \begin{bmatrix} 5 \\ 11 \end{bmatrix}$$

2. **Поток LoRA ($B \cdot A \cdot x$):**
   - Сжатие ($h = A \cdot x$):

$$h = \begin{bmatrix} 0.1 & 0.2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} = 0.1(1) + 0.2(2) = 0.5 \quad (\text{скаляр } 1 \times 1)$$

   - Расширение ($\Delta y = B \cdot h$):

$$\Delta y = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} \cdot 0.5 = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

3. **Суммарный выход (y):**

$$y = y_{\text{orig}} + \Delta y = \begin{bmatrix} 5 \\ 11 \end{bmatrix} + \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 5 \\ 11 \end{bmatrix}$$

4. **Подсчёт ошибки (Loss по функции MSE):**

$$\mathcal{L} = \frac{1}{2} \| y - y_{\text{target}} \|^2 = \frac{1}{2} \left((5-6)^2 + (11-10)^2 \right) = \frac{1}{2}(1+1) = \mathbf{1.0}$$

Шаг 2: Обратный проход (Backward Pass) и градиенты

Нам нужно узнать, как сильно каждый параметр матриц A и B повлиял на ошибку. Базовая матрица W_0 заморожена, поэтому для неё градиенты не считаются.

1. **Градиент по выходу модели ($\frac{\partial \mathcal{L}}{\partial y}$):** Производная MSE ошибки по выходу — это разница предсказания и таргета:

$$\frac{\partial \mathcal{L}}{\partial y} = y - y_{\text{target}} = \begin{bmatrix} 5-6 \\ 11-10 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

2. **Градиент по матрице B ($\frac{\partial \mathcal{L}}{\partial B}$):** По правилу дифференцирования сложных функций:

$$\frac{\partial \mathcal{L}}{\partial B} = \left(\frac{\partial \mathcal{L}}{\partial y} \right) \cdot h^T = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \cdot 0.5 = \begin{bmatrix} -0.5 \\ 0.5 \end{bmatrix}$$

3. **Градиент по матрице A ($\frac{\partial \mathcal{L}}{\partial A}$):** В этот момент срабатывает особенность нулевой инициализации: поскольку матрица B изначально состояла из нулей, производная по A на первом шаге равна **нулю** ($\frac{\partial \mathcal{L}}{\partial A} = 0$).
   - *Почему это важно:* На самом первом шаге градиентного спуска обновляется **только матрица B**. Как только B перестанет быть нулевой, градиенты начнут беспрепятственно поступать и в матрицу A.

Шаг 3: Шаг градиентного спуска (Gradient Descent Update)

Обновляем веса матриц по формуле: $\text{Weight}_{\text{new}} = \text{Weight} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \text{Weight}}$.

1. **Обновление матрицы B:**

$$B_{\text{new}} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} - 0.1 \cdot \begin{bmatrix} -0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.0 - (-0.05) \\ 0.0 - 0.05 \end{bmatrix} = \begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix}$$

2. **Матрица A:** Остаётся без изменений, так как её градиент был равен нулю:

$$A_{\text{new}} = \begin{bmatrix} 0.1 & 0.2 \end{bmatrix}$$

Шаг 4: Проверка (Второй Forward Pass)

Посмотрим, изменилось ли поведение модели после этого единственного микро-шага:

1. $h = A_{\text{new}} \cdot x = 0.5$ (осталось прежним).

2. Новая поправка Δy:

$$\Delta y_{\text{new}} = B_{\text{new}} \cdot h = \begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix} \cdot 0.5 = \begin{bmatrix} 0.025 \\ -0.025 \end{bmatrix}$$

3. Новый выход модели:

$$y_{\text{new}} = \begin{bmatrix} 5 \\ 11 \end{bmatrix} + \begin{bmatrix} 0.025 \\ -0.025 \end{bmatrix} = \begin{bmatrix} 5.025 \\ 10.975 \end{bmatrix}$$

4. **Новый Loss:**

$$\mathcal{L}_{\text{new}} = \frac{1}{2}\left((5.025 - 6)^2 + (10.975 - 10)^2\right) = \frac{1}{2}(0.975^2 + 0.975^2) \approx \mathbf{0.9503}$$

Ошибка уменьшилась с **1.0** до **0.9503**. Всего за один шаг градиентного спуска мини-адаптер начал «подруливать» выход модели в сторону целевого значения, не трогая исходную матрицу W_0.
